Member 2 — Backend and database endpoints

Member 2 owns the public REST API and the database.

Endpoint checklist

Method	Endpoint	Function
GET	/api/health	Check backend health
POST	/api/sessions	Create a session
GET	/api/sessions/{session_id}	Retrieve session details
POST	/api/sessions/join	Join a session
GET	/api/sessions/{session_id}/participants	List participants
POST	/api/sessions/{session_id}/interpretations	Save an interpretation
GET	/api/sessions/{session_id}/status	Retrieve session and analysis status
POST	/api/sessions/{session_id}/analyze	Trigger AI analysis
GET	/api/sessions/{session_id}/issues	Retrieve analysis findings
PATCH	/api/issues/{issue_id}	Resolve or dismiss an issue
POST	/api/sessions/{session_id}/agreement	Generate/save an agreement
POST	/api/agreements/{agreement_id}/confirm	Record participant confirmation
GET	/api/sessions/{session_id}/bridge-test	Calculate Bridge Test metrics

Backend implementation order

First: Session workflow

1. Create session.

2. Join session.

3. Get session details.

4. Save interpretations.

Second: AI integration

5. Trigger analysis.

6. Call Member 3's AI module.

7. Validate its response.

8. Save analysis and issues.

9. Return issues to the frontend.

Third: Resolution workflow

10. Update issue status.

11. Generate the shared agreement.

12. Record confirmations.

13. Calculate Bridge Test metrics.

Database tables

Member 2 should create these tables:

Table	Purpose
sessions	Task details and session state
participants	Session participants
interpretations	Independent responses
analyses	AI analysis runs and results
issues	Potential misunderstandings and their resolutions
agreements	Shared checklist and agreement status
agreement_confirmations	Participant confirmations

Suggested backend folder structure

```
backend/
├── app/
│   ├── main.py
│   ├── database.py
│   ├── models.py
│   ├── schemas.py
│   ├── routers/
```

```
│   │   ├── sessions.py
│   │   ├── interpretations.py
│   │   ├── analysis.py
│   │   ├── issues.py
│   │   └── agreements.py
│   └── services/
│       ├── session_service.py
│       ├── analysis_service.py
│       └── agreement_service.py
├── requirements.txt
└── .env
```

Member 2 should expose FastAPI's Swagger documentation at /docs during development. This lets Member 1 test endpoints without waiting for the frontend to be finished.

Member 2's final deliverables

- [ ] FastAPI application
- [ ] Database schema
- [ ] All agreed REST endpoints
- [ ] Input validation
- [ ] Session and participant access checks
- [ ] AI module integration
- [ ] Issue persistence and resolution
- [ ] Agreement confirmation
- [ ] Bridge Test calculations
- [ ] Deployed backend